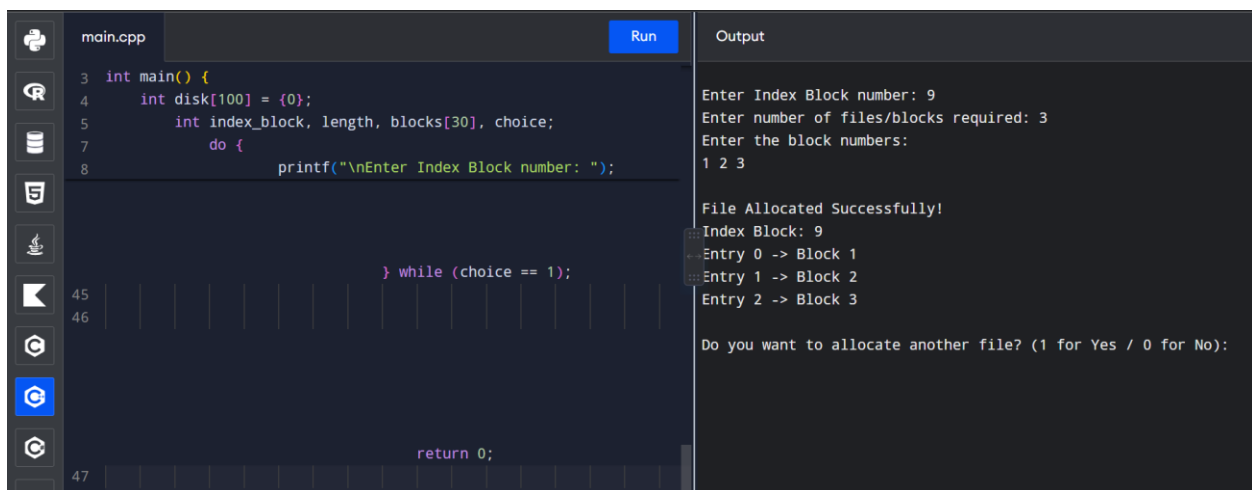


EXPERIMENT 29:

29. Consider a file system that brings all the file pointers together into an index block. The i th entry in the index block points to the i th block of the file. Design a C program to simulate the file allocation strategy.

AIM:

To design a C program to simulate the Indexed File Allocation strategy.



The screenshot shows a C program in a code editor with a dark theme. The code is in a file named 'main.cpp'. It includes a 'Run' button and a 'Output' pane. The code defines an array 'disk' of size 100, an 'index_block' array of size 30, and a 'choice' variable. It uses a 'do' loop to prompt the user for an 'Index Block number'. The output pane shows the following sequence of events: the user enters '9' for the index block number, then '3' for the number of files/blocks required, then '1 2 3' for the block numbers. The program outputs 'File Allocated Successfully!', 'Index Block: 9', and then maps 'Entry 0' to 'Block 1', 'Entry 1' to 'Block 2', and 'Entry 2' to 'Block 3'. Finally, it asks 'Do you want to allocate another file? (1 for Yes / 0 for No):'.

```
main.cpp
3 int main() {
4     int disk[100] = {0};
5     int index_block, length, blocks[30], choice;
6     do {
7         printf("\nEnter Index Block number: ");
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
return 0;
}
```

Run

Output

Enter Index Block number: 9
Enter number of files/blocks required: 3
Enter the block numbers:
1 2 3

File Allocated Successfully!
Index Block: 9
Entry 0 -> Block 1
Entry 1 -> Block 2
Entry 2 -> Block 3

Do you want to allocate another file? (1 for Yes / 0 for No):

PSEUDO CODE :

BEGIN

PRINT "Enter total blocks on disk:"

READ total_blocks

DECLARE disk array initialized to 0

LOOP

PRINT "Enter Index Block number:"

READ index_block

IF disk[index_block] == 1 THEN

PRINT "Index block already allocated! Choose another."

CONTINUE

END IF

PRINT "Enter number of blocks required for file:"

READ length

DECLARE blocks_requested array of size length

PRINT "Enter block numbers for index entry:"

SET conflict = 0

FOR i FROM 0 TO length - 1 DO

 READ blocks_requested[i]

 IF disk[blocks_requested[i]] == 1 THEN

 SET conflict = 1

 END IF

END FOR

IF conflict == 1 THEN

 PRINT "Allocation failed: One or more blocks are already in use."

ELSE

 SET disk[index_block] = 1

 FOR i FROM 0 TO length - 1 DO

 SET disk[blocks_requested[i]] = 1

 END FOR

 PRINT "Allocated Index Block: " + index_block

 FOR i FROM 0 TO length - 1 DO

 PRINT "Index Entry [" + i + "] -> Disk Block " + blocks_requested[i]

 END FOR

END IF

PRINT "Do you want to add another file? (1-Yes, 0-No):"

```
    READ choice
    UNTIL choice == 0
END
```

CODE:

```
#include <stdio.h>

int main() {

    int disk[100] = {0};

    int index_block,
length, blocks[30],
choice;

    do {

        printf("\nEnter
Index Block number: ");

        scanf("%d",
&index_block);

        if
(disk[index_block] ==
1) {
```

```
        printf("Error:  
Block %d is already  
assigned as an index or  
data block.\n",  
index_block);
```

```
    } else {
```

```
        printf("Enter  
number of files/blocks  
required: ");
```

```
        scanf("%d",  
&length);
```

```
        printf("Enter the  
block numbers:\n");
```

```
        int conflict = 0;
```

```
        for (int i = 0; i <  
length; i++) {
```

```
            scanf("%d",  
&blocks[i]);
```

```
            if  
(disk[blocks[i]] == 1) {
```

```
        conflict = 1;
```

```
    }
```

```
}
```

```
    if (conflict == 1)
```

```
{
```

```
    printf("Allocation
```

```
    Failed: One or more
```

```
    blocks are already
```

```
    occupied.\n");
```

```
    } else {
```

```
    disk[index_block] = 1;
```

```
        for (int i = 0; i
```

```
< length; i++) {
```

```
    disk[blocks[i]] = 1;
```

```
    }
```

```
        printf("\nFile  
Allocated  
Successfully!");
```

```
printf("\nIndex Block:  
%d\n", index_block);
```

```
        for (int i = 0; i  
< length; i++) {
```

```
printf("Entry %d ->  
Block %d\n", i,  
blocks[i]);
```

```
        }
```

```
    }
```

```
}
```

```
printf("\nDo you  
want to allocate another  
file? (1 for Yes / 0 for  
No): ");
```

```
scanf("%d",
```

```
&choice);
```

```
    } while (choice == 1);
```

```
    return 0;
```

```
}
```